Optimize Anything — Technical Report

Ridhima Trivedi · 241071062 · TY BTech CS, Batch D · Optimization Techniques, Assignment 8 Repository: this folder · All tables: `docs/RESULTS_SUMMARY.md` (generated from `results/`) · AI disclosure: `docs/AI_TOOLS.md` · Failures: `docs/FAILURE_LOG.md`

> **Reading guide.** Each track follows the Section 4 workflow: process → baseline → formulation → method choice → results → correctness check → trade-offs. "Measured" means timed on the test machine (2 vCPUs); "simulated" means a model or a quantum simulator; "theoretical" means derived, not measured. Stochastic results are mean ± std over 3–10 seeds.

---

1. Track E — Main case study: hospital nurse rostering

1.1 Process and evidence it needs optimisation

A ward in-charge builds a 14-day roster for 10–80 nurses over three shifts (Morning 07–14, Evening 14–21, Night 21–07). Done by hand — filling each shift from the top of the staff list — the *baseline* roster for a 20-nurse ward leaves on average **13 shift-slots understaffed, 12.8 shifts with no senior nurse, and 36 approved day-off requests ignored** (5 synthetic instances; generator parameters in `trackE/nurse.py::GEN`, demand ≈ 97% of contracted capacity, as on a busy ward). Those are patient-safety and staff-retention problems, not just inconvenience.

1.2 Formulation

Decision variables x[n,d,s] ∈ {0,1} (nurse n works shift s on day d).

Hard constraints	Soft objective (minimise, weights in brackets)
H1 ≤ 1 shift per nurse per day	S1 understaffing, per missing nurse [100]
H2 no Morning/Evening the day after a Night (rest)	S2 shift without a senior nurse [60]
H3 contract: full-time ≤ 10, part-time ≤ 6 shifts / 14 days	S3 day-off request worked [10]
H4 ≤ 3 consecutive nights	S4 night fairness: max − min nights among full-timers [5]
H5 approved leave never worked	S5 overstaffing, per extra nurse [2]

The ILP linearises S1/S5 with `under[d,s] − over[d,s] = demand − staff`, S2 with a binary `nosenior[d,s] ≥ 1 − Σ seniors`, and S4 with `nmax ≥ nights_n ≥ nmin`. Success metric = total penalty from one **independent checker** (`evaluate()`), plus hard violations (must be 0) and runtime.

1.3 Strategy selection

Method	Variable type	Exact?	Size it handles here	Hardware	Parallel?	Role
Manual rule (baseline)	binary	no	any, instant	—	no	what wards do now
Greedy with scoring	binary	no	any, < 0.2 s	1 core	no	fast first roster
Simulated annealing	binary	no	any, ≈ 2 s	1 core	could run chains in parallel	improve greedy
ILP (HiGHS branch-and-cut)	binary + continuous	**yes** (proves optimality)	3,500 vars / 4,400 rows in ≈ 9 s	1 core	solver-internal	best quality
Rejected: gradient methods	—	—	—	—	—	variables are discrete; relaxation + rounding breaks H2/H4
Rejected: QAOA / annealer	—	—	—	—	—	560–3,360 binary variables far exceed simulators and embeddable QUBO size (Track I)

1.4 Results (mean of 5 instances per ward size; full table in the summary)

Nurses	Manual	Greedy	Sim. annealing	ILP (time)
10	1,996	714	291	**147** (0.8 s)
20	2,458	448	186	**37** (1.7 s)
40	4,179	911	413	**8** (3.5 s)
80	7,659	1,944	1,205	**52** (9.3 s)

The ILP proved optimality on all 20 instances. For the 20-nurse ward it removes all understaffing and all no-senior shifts and

honours all but 3.6 of ~36 requests, with nights spread within 0.2 shifts. Annealing roughly halves greedy's penalty but stalls because its moves cannot trade one nurse's shift for another's across days.

Correctness. The ILP's objective is asserted equal to the independent checker's cost on every instance; every method has 0 hard violations (the checker found a bug in the annealing feasibility test — failure #8).

Sensitivity and limits. With demand +25% (flu season) the manual roster is 47 slots short and the ILP still 46 short: optimisation cannot create capacity. The model's value then is to quantify the staffing gap. Limits: synthetic data; equal-weight objective (weights are a management decision); no individual skill matrix beyond senior/junior; real wards need 4–6-week horizons (≈ 2–3× variables; still within ILP reach given the scaling observed).

---

2. Track A — Continuous optimisation

Objective. Compare GD, SGD, mini-batch SGD, Momentum, NAG, AdaGrad, RMSProp, Adam, Newton and Gauss–Newton on landscapes chosen to expose their differences, and on an ML task. All optimisers are written from scratch (`trackA/optimizers.py`); learning rates tuned by grid search with the same iteration budget for every method.

Findings.

- *Strongly convex quadratic (κ = 50):* NAG 67 iterations to 1e-6, Momentum 132, Adam 125, GD 214, Newton 1. GD with lr = 1/L stays below the theoretical (1 − μ/L)^2k curve at every step (asserted).
- *Convex but not strongly convex (Σx⁴):* GD's log-log slope is −1.95 (theory −2) — sublinear, as predicted when curvature vanishes at the optimum.
- *Non-convex Rosenbrock:* Newton 5 iterations, Momentum 394; GD, AdaGrad and RMSProp do not reach 1e-6 in 5,000.
- *Saddle point:* first-order methods escape to the minima; **plain Newton converges to the saddle** (Hessian eigenvalues −2, 2), saddle-free Newton (|H|) escapes. *Local minima:* 52–62% of starts reach the global minimum; momentum helps slightly.
- *Noise:* with gradient noise σ = 2, a decaying step $1/(\mu(k + k_0))$ reaches f ≈ 0.002, ten times better than constant steps.
- *ML (logistic regression):* at equal epochs, mini-batch and adaptive methods reach f − f* ≈ 2–4e-4; full-batch GD 2.4e-3. Test accuracy ≈ 0.96 for all — optimisation precision beyond this does not change generalisation.
- *Backprop* for a 65-parameter MLP matches finite differences to 4.5e-10; the same gradient is fed to four optimisers.
- *Curvature:* exact Newton, Newton-CG with Hessian-vector products (112 HVPs total) and natural gradient all reach 1e-8 in 7 iterations vs 1,638 for GD; diagonal preconditioning 96 (after step tuning), top-5 low-rank Newton 27. The model Fisher equals the Hessian for logistic loss; the empirical Fisher differs by 74% and converges much worse.
- *Gauss–Newton* fits a 3-parameter exponential in 7 iterations (Adam 273, GD 505).

Trade-off. Second-order methods win per iteration but cost O(d³) per step (or many HVPs); for large models diagonal/low-rank/Adam-style approximations are the practical middle ground.

3. Track B — Discrete optimisation

- *Knapsack (n = 20–200):* DP, ILP and branch-and-bound always agree (asserted); greedy is within 1.1% at n = 20 and 0.03% at n = 200. DP is fastest here (capacity is small); ILP has the largest fixed overhead.
- *Makespan:* LPT is 2.5–4% above optimal and always within Graham's 4/3 − 1/(3m) bound; our B&B needs ~970,000 nodes (2.8 s) at n = 20 versus 0.14 s for the ILP — exponential growth of plain enumeration.
- *Max-Cut QUBO/Ising:* the energy of every one of the 2^8 assignments is identical in the direct cut, QUBO and Ising forms (mismatch 0.0). Local search finds the optimum 55% of the time, SA 100%. This instance is reused in Tracks C/I.
- *Penalty QUBO (choose exactly 6 features):* P ≤ 0.5 returns infeasible subsets of size 3–5; P ≥ 1 returns the constrained optimum — consistent with the rule "P must exceed the largest single gain (0.78)".

4. Track C — Quantum optimisation (simulated)

- *QAOA on Max-Cut:* expected-cut approximation ratio 0.72–0.84 depending on depth and optimiser (COBYLA cheapest at ~50–200 circuit evaluations; parameter-shift GD 2,000–6,000). **The classical baseline wins**: SA finds the optimum every time, local search averages 0.94. QAOA's best *sampled* cut is optimal, but the expected cut is not.
- *Shots:* estimator std follows 1/√shots (std·√shots ≈ 1.6–2.2 constant); training with 64 shots drops the ratio to 0.69.
- *Noise:* the ratio falls from 0.84 (noiseless) to 0.72 at 5% two-qubit depolarising error — approaching random (0.64).
- *VQE (H_2):* QNG reaches chemical accuracy in 4 iterations, parameter-shift GD 18, COBYLA 45; SPSA only 2/5 runs in 100 iterations (cheapest per step: 2 evaluations regardless of parameter count).
- *Barren plateau:* gradient variance of a global cost decays ≈ 2^(−0.89n) with qubits; a local cost decays much more slowly — cost-function design matters more than the optimiser.

Simulation vs hardware: everything is exact or sampled simulation; the noise model is synthetic (Track I uses a device-calibrated model).

5. Track D — Submodular, parallel and distributed

- *Submodular:* weighted coverage passes 3,000 random diminishing-returns tests (0 violations), so greedy's (1 − 1/e) guarantee applies; observed greedy/OPT ≥ 0.94. Lazy greedy gives identical answers with 494 instead of 7,225 oracle calls. A complementary (pair-bonus) objective fails the test (209 violations) and no guarantee is claimed.
- *Load balancing (measured, pthreads, heavy-tailed sorted tasks):* static block split reaches only 1.1× on 2 cores (one thread gets the largest tasks, imbalance 1.77); cyclic, central dynamic queue and work stealing reach ≈ 2.0×. Simulation to 16 workers: block 2.8×, stealing 14.7×.
- *Amdahl/Gustafson:* the Karp–Flatt serial fraction from 2 cores is within timing noise of zero (repeat runs gave −0.016 and +0.016), so it cannot be extrapolated. For an assumed serial fraction of 5%, Amdahl caps 16 cores at 9.1× while Gustafson's scaled speedup is 15.3× (**model predictions, not measurements** — only 2 cores were available).
- *Locality:* loop order ikj is 1.4–4.4× faster than ijk (sequential access of B's rows); 64-blocking helps less here.
- *Synchronisation:* a mutex per increment makes 2 threads 3–4× *slower* than 1 (lock ping-pong); atomics are 3× faster than the mutex; thread-local accumulation removes the cost entirely.
- *Communication:* with 1 KB–10 MB arguments and tiny tasks, a 2-process pool is 2–4,000× slower than serial — pickling dominates; only coarse tasks with small payloads benefit.
- *Data/model/pipeline parallelism (NumPy MLP):* data-parallel gradients equal the full-batch gradient (7e-13) with 1.69× on 2 processes; tensor-parallel outputs are exact (1.18×); a 2-stage process pipeline is slower than serial (queue serialisation), while the GPipe model shows the bubble (S−1)/(M+S−1) shrinking with more micro-batches.
- *FedAvg (10 clients):* more local steps E cut communication (IID: 37 → 15 rounds from E = 20 → 50; E ≤ 5 never reach the target in 60 rounds). Label-skewed clients plateau ~15× higher (client drift).
- *Multi-task:* GradNorm gives the lowest mean relative loss; MGDA's min-norm direction favours one task and leaves the large-scale task worse; equal weighting is in between. No method dominates on both tasks.

6. Track F — Compiler optimisation

Build. MiniC: lexer, recursive-descent parser, three-address IR, an IR virtual machine (instruction count and a cycle model), and a C back end compiled with `gcc -O0` (so gcc does not redo our optimisations). **Optimisations (6):** global constant propagation and folding (dataflow over the CFG, including constant branches), copy propagation, local CSE (value numbering), liveness-based DCE + unreachable code, loop-invariant code motion over natural loops, strength reduction/algebraic simplification. Plus Chaitin–Briggs and ILP register allocation.

Results. Dynamic IR instructions fall 20–60% on all five benchmarks and static size by 30–60%; outputs are identical in the VM and natively. Native speedups under gcc -O0 range from 0.90× to 1.95× — `sum_invariant` is *slower* (0.84–0.90× across runs) even though it executes 40% fewer IR instructions, because at -O0 every variable is a stack slot and the loop is limited by the store→load chain that both versions share (a measurement-validity lesson). Ablation: LICM is the single most valuable pass; removing DCE costs most within the pipeline.

Formulated decisions.

1. *Pass ordering:* choose 4 passes from 6 (1,296 sequences) to minimise total cycles. Exhaustive search finds 753,981 cycles; greedy finds 777,888 (3% worse) with 24 instead of 1,296 evaluations. The textbook order (constprop, copyprop, cse, sr) is poor here (892,015) because it omits LICM.
2. *Register allocation:* minimise loop-weighted spill cost subject to K-colouring. On our programs Chaitin–Briggs is already optimal (ILP proves it). On random interference graphs Chaitin spills 2–6× more cost; the ILP costs 0.6–4.5 s vs 0.2 ms.

Correctness: differential fuzzing — 1,000 random programs × 3 pass pipelines + register allocation, 0 mismatches.

7. Track G — xv6 operating system (QEMU, RISC-V)

Features (patch `trackG/xv6-a8.patch` against upstream `e93dabe`):

1. **MLFQ scheduler** (runtime-selectable; round robin remains the default): 3 levels with quanta 1/2/4 ticks, demotion on quantum exhaustion, priority boost every 50 ticks, preemption when a higher-priority process becomes runnable. *OS concept:* scheduling policy, the turnaround/response trade-off, starvation.
2. **Copy-on-write fork** (runtime-selectable): shared read-only pages with a `PTE_COW` bit, per-page reference counts in the allocator, a store-page-fault handler, and COW-aware `copyout`. *OS concept:* virtual memory, page faults, lazy work.
3. Syscalls `setsched`, `waitstat` (per-process creation/first-run/exit/run ticks), `setcow`, `memstat`.

Results (1 CPU, 3 fresh boots per configuration; ticks ≈ 0.1 s). *COW:* fork+exit with a 2 MB heap takes 1.1 ms instead of 17.9 ms (16×) and copies 0 instead of 10,340 pages per 20 forks; 8 live children use 57 instead of 4,192 pages (−98.6%). The trade-off is real: when the child writes 100% of the heap, COW is 1.9× *slower* (37 vs 19 ms — a fault plus a copy per page instead of one bulk copy); break-even is ≈ 50% of pages written. *MLFQ:* short jobs arriving behind long jobs (W3) finish in 0.1 instead of 2.1 ticks with zero response time; interactive jobs sharing the CPU with hogs (W2) finish 41% sooner (13.0 vs 22.0 ticks). The cost: an all-CPU-bound batch (W1) has 11% worse average turnaround (12.6 vs 11.3) because demoted jobs wait behind each other. Before the preemption fix (failure #9) W2 was 51% *worse* than round robin (`results/trackG/sched_bench_BEFORE_preemption_fix.json`).

Correctness: an isolation test (parent/child/grandchild and a pipe write into a shared page never leak); `usertests`**: ALL TESTS**

PASSED (65 tests, 3 CPUs) both with COW + MLFQ switched on and with both off (`trackG/usertests_*.log`). The first full run exposed a lock-order deadlock in the statistics code (failure #13) that the 1-CPU benchmarks never hit.

8. Track H — DBMS (PostgreSQL 16, TPC-H SF 0.1)

Workload: 8 weighted OLTP-style lookups of an order-management application + TPC-H Q3, Q5, Q6, Q10, Q12, Q14. **Formulation:** choose $x_i \in \{0,1\}$ over 16 candidate indexes (from WHERE/JOIN/ORDER BY columns) to minimise $\Sigma_q w_q \cdot \text{cost}(q \mid \text{indexes})$ subject to $\Sigma\, \text{size}_i \cdot x_i \le B$. Cost oracle: PostgreSQL's planner with HypoPG hypothetical indexes ("what-if"), so candidates are evaluated without building them. **Methods:** default (primary keys only), greedy (best what-if saving per byte), ILP (atomic-benefit model, HiGHS), and "all candidates" as the unlimited-storage reference. **Results (measured after building each configuration; median of 15 warm runs, two independent measurement runs).** The weighted workload falls from ≈ 3.0 s (primary keys only) to ≈ 0.49 s with an 11 MB index budget (≈ 6×), close to the 48 MB "all candidates" result. At the tightest budget (10%) the ILP is ≈ 10% faster than greedy in both runs (2.22 vs 2.47 s) and has the lower what-if cost: ratio-greedy spends the budget on small indexes, the ILP buys the one large `lineitem(l_partkey)` index. At 25% both choose the same indexes, and at 50% their difference is within run-to-run noise (±10%). An earlier 5-run measurement showed the 10% ordering *reversed* — the reason for 15 runs and two repeats (`results/trackH/index_selection_run1.json`). Lesson: planner cost is a good but imperfect oracle for wall-clock time. **Correctness:** every configuration returns the same row multiset for every query (exact order where `ORDER BY` defines it — failure #12 shows why that distinction matters). `EXPLAIN ANALYZE` plans are saved (Seq Scan → Bitmap Index Scan).

9. Track I — Quantum algorithms and QML vs classical (simulated)

All circuits are hand-written: Deutsch–Jozsa (1 query vs 17 classical worst case, n = 5), Bernstein–Vazirani (1 vs 6), Grover, QFT (matches the DFT matrix to 4e-15), QPE (error halves with each counting qubit), Shor (factors 15 for a = 7, 2, 4, 11, 13), and quantum annealing (exact adiabatic simulation of the Track B Max-Cut: P(optimum) 0.01 at T = 0.5 → 0.997 at T = 40, governed by the minimum spectral gap).

- **Grover vs linear search:** query counts follow √N vs N/2 (25 vs ~512 at N = 1,024), but *simulating* Grover takes longer than the classical search at every size and grows exponentially — a query advantage is not a wall-clock advantage without hardware. On the FakeManilaV2 device noise model, Grover (n = 3, 37 two-qubit gates after routing) succeeds with P ≈ 0.55 instead of 0.94.
- **Annealing hardware limits:** our 8-node Max-Cut embeds easily, but complete graphs need chains of physical qubits; larger K_n need many qubits per logical variable or fail on Chimera — dense QUBOs (e.g. Track E) do not fit.
- **QML vs classical (moons, 5 splits):** the variational classifier (12 parameters) reaches 93.3 ± 7.0% test accuracy — better than logistic regression (88.3%) and a small MLP (86.7%) but below an RBF-SVM (97.2%), which trains in about a millisecond versus 22 s and ~126,000 circuit evaluations. The quantum-kernel SVM reaches 77.8%. Net: no quantum advantage on this classical low-dimensional data; on hardware each evaluation would also cost many shots.
- **Contrast with Track D:** parallel speedups are *measured* today (≈ 2× on 2 cores) and bounded by Amdahl; quantum speedups here are *theoretical query counts*, and simulation is exponentially slower.

10. Track J — Software engineering: test-suite minimisation (more-itertools)

Subject: `more-itertools` (7,282 library lines, 769 tests, ~21,000 subtests; public OSS project). **Profile:** full suite 17.7 s wall. Per-test times come from the coverage-instrumented run and are used as relative weights. **One test (`PrimeFunctionTests.test_primes`) accounts for 65% of the time**, the next one 13%. **Formulation:** weighted set cover — $x_t \in \{0,1\}$, minimise $\Sigma\, \text{time}_t \cdot x_t$ such that every line covered by the full suite stays covered. **Methods:** greedy (most new lines per second), ILP (HiGHS, exact), random subset of the same size. **Results:** both minimisers keep 100% line coverage with ~65% fewer tests (ILP 266, greedy 291 of 769), whereas a random subset of 266 keeps only 69.5% coverage. But **measured run time does not fall** (18–21 s vs 17.7 s): the slow tests cover lines no other test covers, so every coverage-preserving suite must include them. The formulation optimised the test *count*, while the bottleneck is one irreplaceable slow test — the next target is that test itself (or a coverage criterion at function level). *Fault detection:* of 40 AST mutants, 28 are killed by the full suite; the reduced suites kill 78.6% of those, a random subset 67.9% — 100% line coverage does not preserve assertions. *Prioritisation:* ordering by additional coverage per second gives APFD 0.78 vs 0.51 for the original order.

---

11. Answers to the final questions (Section 11)

1. **Process and evidence.** Nurse rostering (Track E): the manual-style roster leaves 13 slots understaffed, 12.8 shifts without a senior nurse and ~36 requests ignored per 20-nurse ward fortnight.
2. **Formulation.** Binary x[n,d,s]; five hard constraints; weighted soft penalties linearised with auxiliary variables (§1.2). Each domain track states its own formulation (index selection = knapsack-like ILP, pass ordering = combinatorial search, register allocation = weighted colouring ILP, test minimisation = weighted set cover).
3. **Methods and why.** Discrete, constrained, mid-sized problems → ILP for proven optimality, greedy/annealing for speed (Tracks B, E, F, H, J). Smooth objectives → gradient and curvature methods (A). Monotone submodular → greedy with a checked guarantee (D). Quantum methods were tested where they apply (C, I) and rejected for Track E.

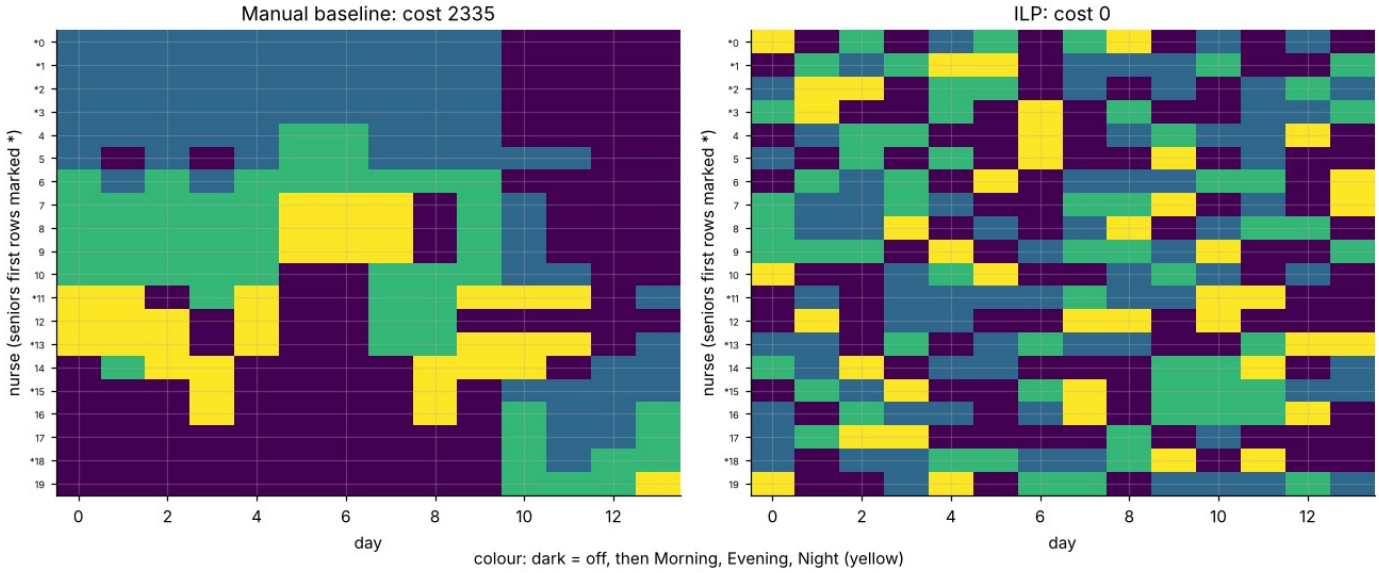
4. **Optimised vs baseline.** Rostering penalty −98.5% (20 nurses, ILP); compiler IR instructions −20 to −60%; COW fork 16× faster and 98.6% less memory; DBMS workload ≈ 6× faster; MLFQ interactive turnaround −41% and short-job turnaround 2.1 → 0.1 ticks; test suite −65% tests at equal coverage (but not faster).
5. **Trade-offs, failures, limits.** ILP time grows with size; COW loses when children write most pages; MLFQ needs preemption and boosting; a lower planner cost need not be faster; coverage-preserving minimisation does not cut time when slow tests are irreplaceable; MLFQ costs CPU-bound batches 11%; QAOA and QML lost to the best classical baselines; fifteen AI-generated bugs (failure log).
6. **Correctness and reproducibility.** Independent checkers and assertions (exact methods agree; ILP = checker cost), differential fuzzing, usertests on 3 CPUs, identical SQL result sets, gradient checks, fixed seeds, `run_all.sh`, unit tests, and auto-generated tables.
7. **Changing scale/data/hardware.** Rostering: longer horizons → ILP with time limits or rolling horizons; very large wards → annealing/large-neighbourhood search seeded by greedy. DBMS: larger workloads → greedy with what-if calls is cheaper than enumerating ILP benefits. Parallel: Amdahl predicts diminishing returns past ~16 cores for our serial fraction. Quantum: only fault-tolerant hardware could turn query advantages into speed.
8. **Compiler / xv6 / DBMS concepts.** Compiler: dataflow analysis, dominators/natural loops, interference graphs — correctness shown by fuzzing. xv6: scheduling policy (starvation, preemption) and virtual memory (page faults, reference counting, kernel `copyout`) — correctness by usertests and an isolation test. DBMS: access paths and the cost-based optimiser — correctness by comparing result multisets.
9. **Quantum vs parallel.** Quantum algorithms give provable query reductions (DJ, BV, Grover) but, simulated, run slower; the variational classifier beat linear and small-MLP baselines but not an RBF-SVM, at ~10^5 circuit evaluations; noise and barren plateaus limit scaling. Parallel computing gave measured ≈ 2× speedups on 2 cores, limited by load imbalance, synchronisation and communication.
10. **AI tools.** Claude Opus 5.5 for everything (register in `docs/AI_TOOLS.md`). Fifteen AI errors were found and fixed, including a kernel deadlock, a missing MLFQ rule, an infeasible ILP from a data-join bug, and an inverted classifier. The second-tool comparison is to be completed by the student (`docs/TOOL_COMPARISON.md`).

12. Limitations and next steps

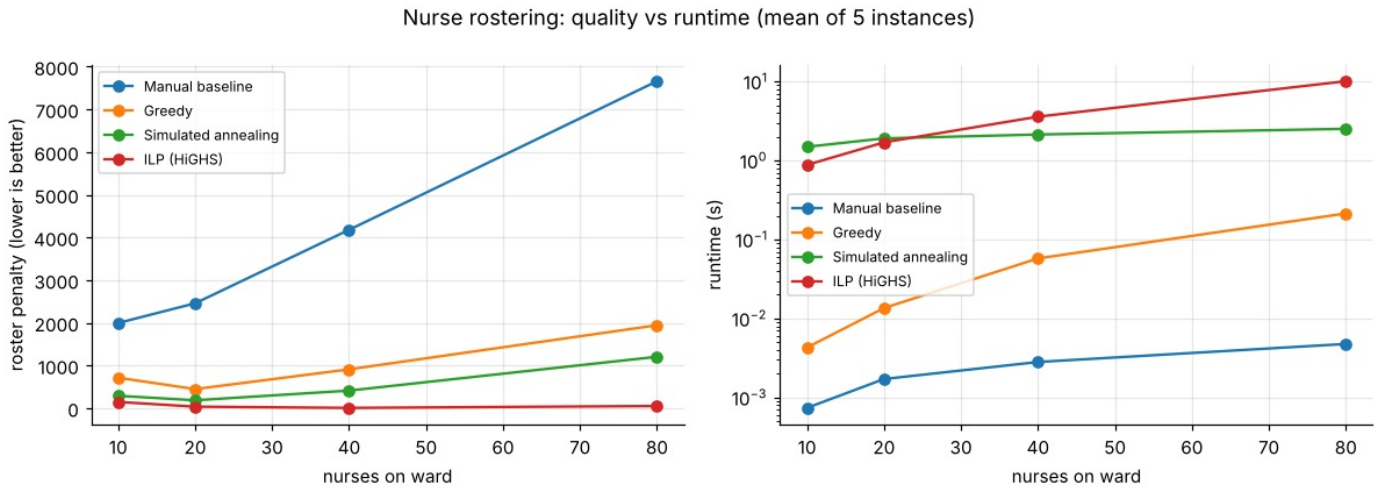
- Only 2 CPU cores: parallel scaling beyond 2 is simulated or predicted.
- No real quantum hardware; noise models stand in for devices.
- Track E data are synthetic; next step is a real ward's demand and leave data, and a 4–6-week horizon.
- Track G benchmarks use one CPU for comparability; multi-CPU MLFQ needs per-CPU queues.
- Track H uses SF 0.1; a larger scale factor would increase index benefits and build costs.
- Track J should next target the slow, irreplaceable tests directly.

Appendix A — Key figures

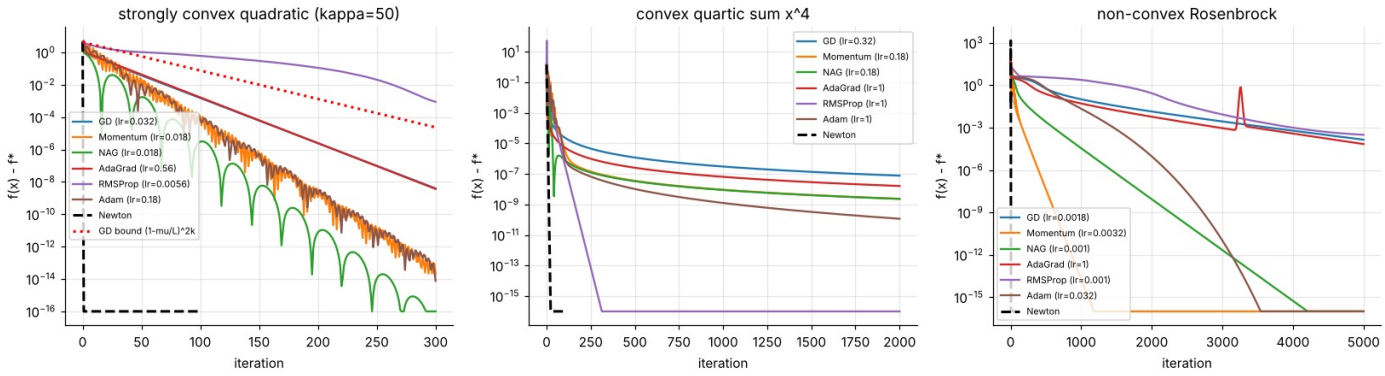
E_rosters



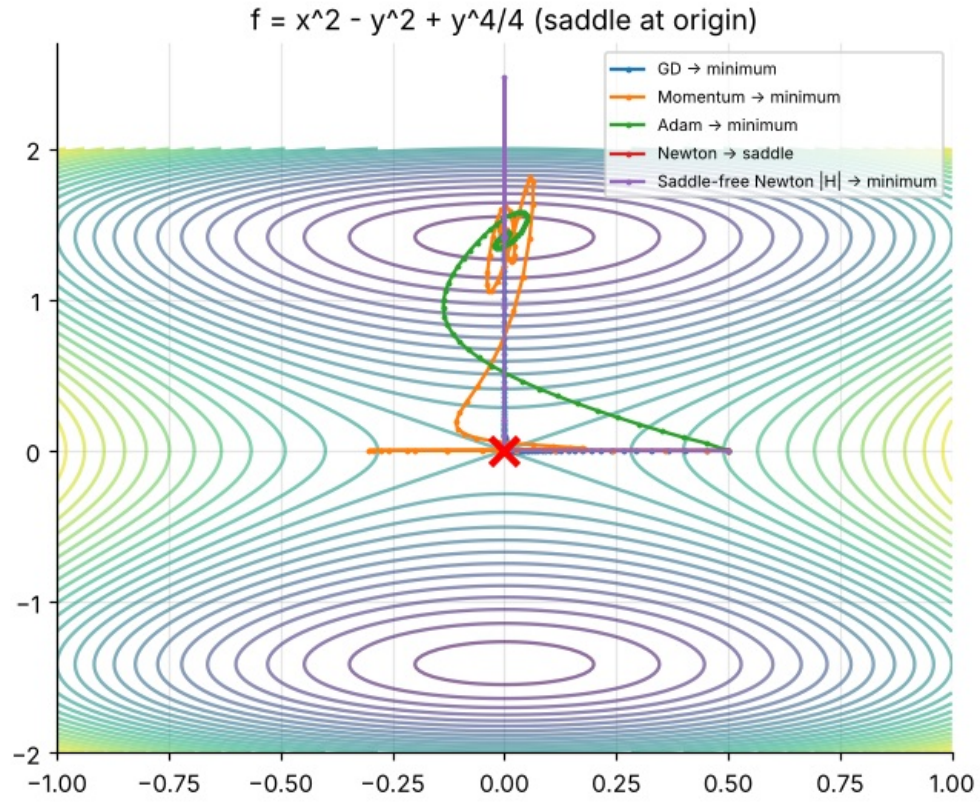
E_quality_runtime



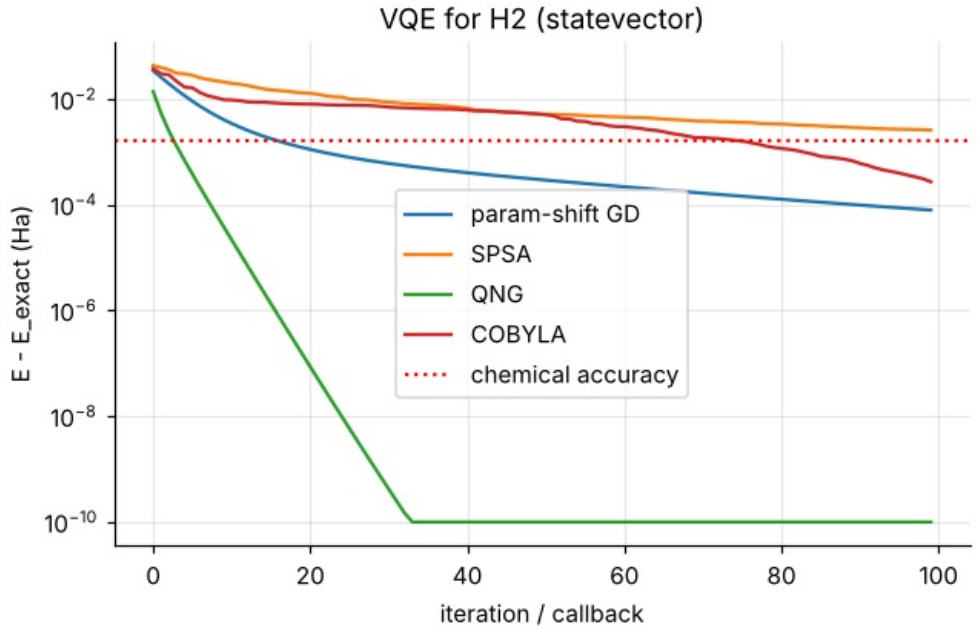
A1_convergence



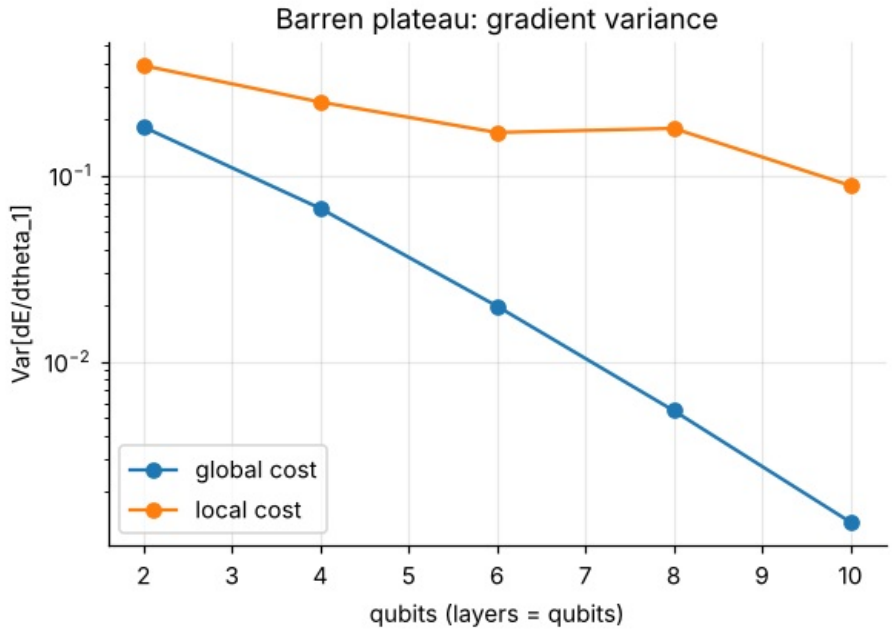
A2_saddle



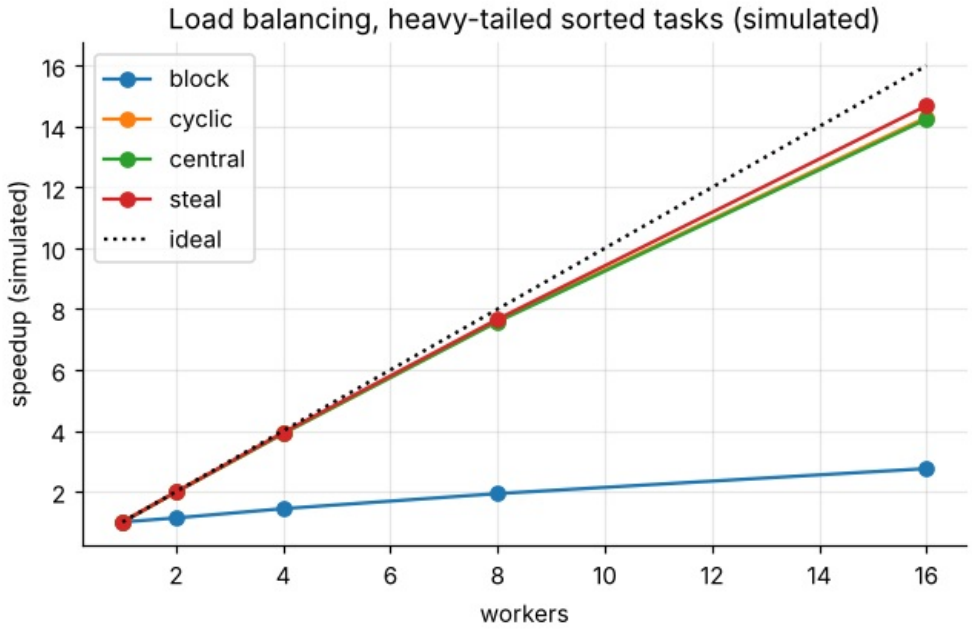
C4_vqe



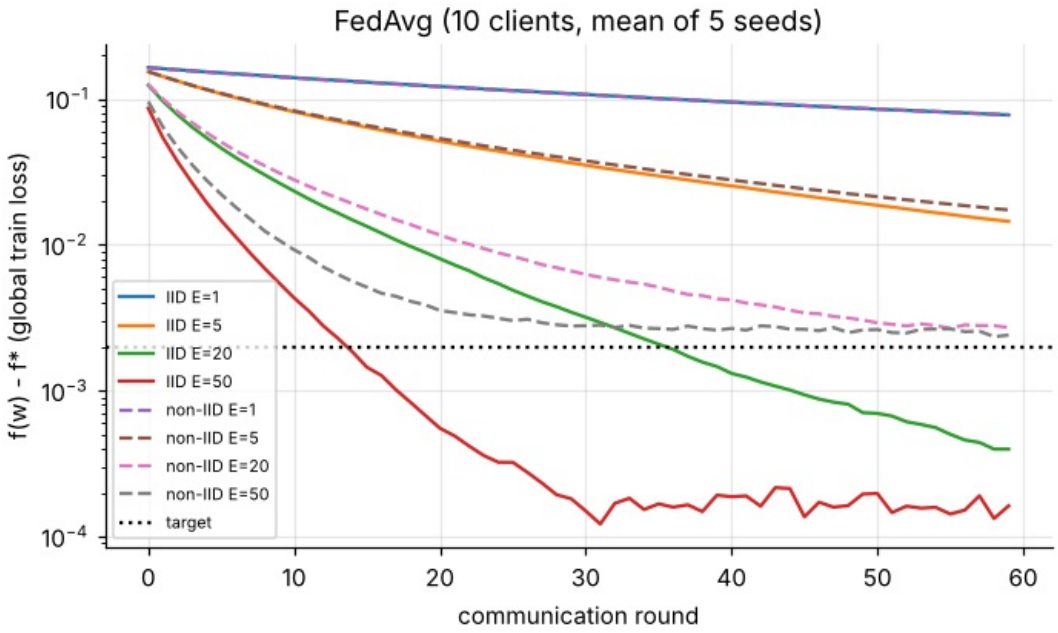
C5_barren



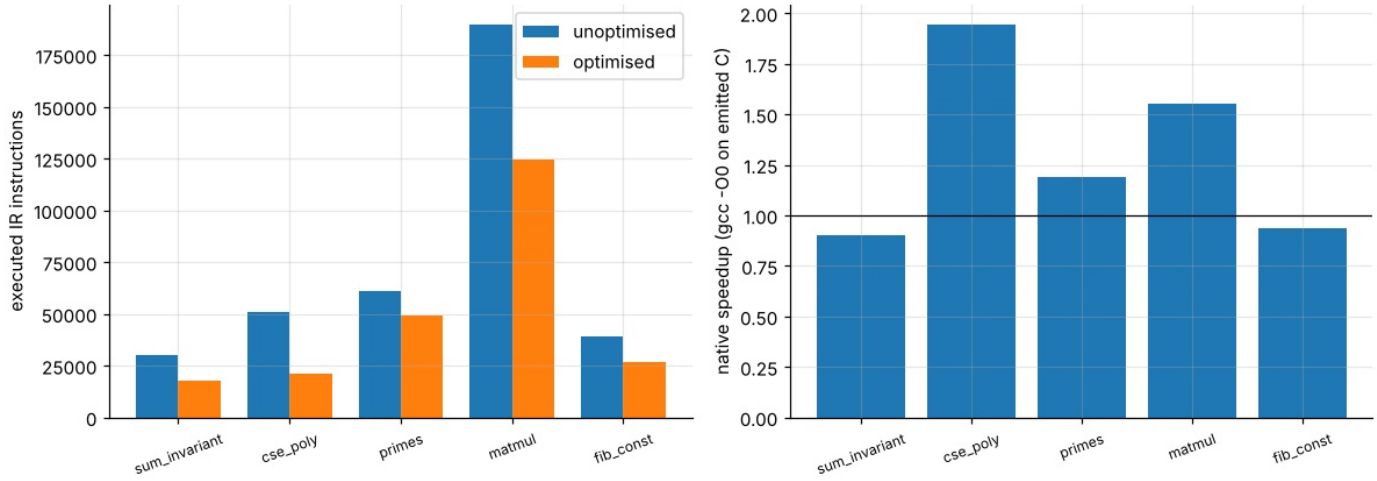
D_sched_sim



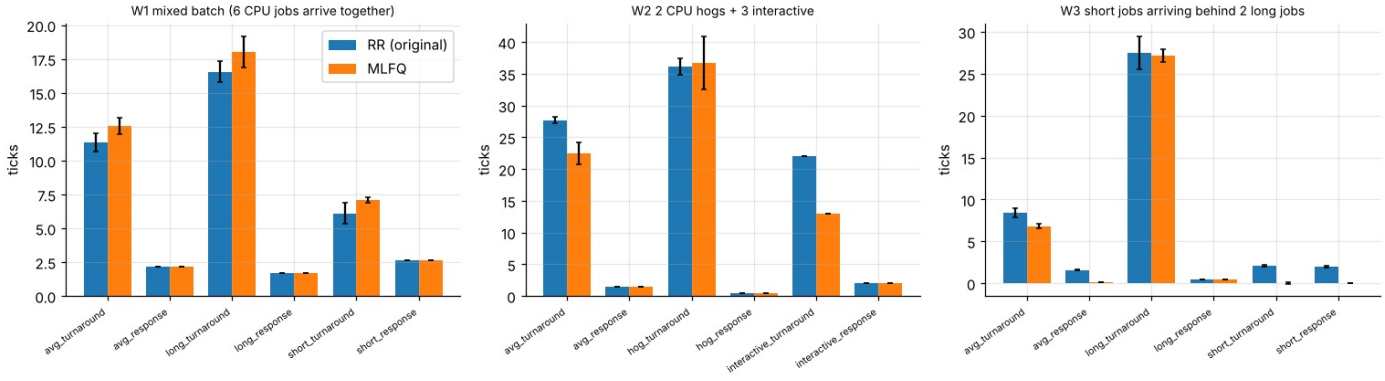
D_fedavg



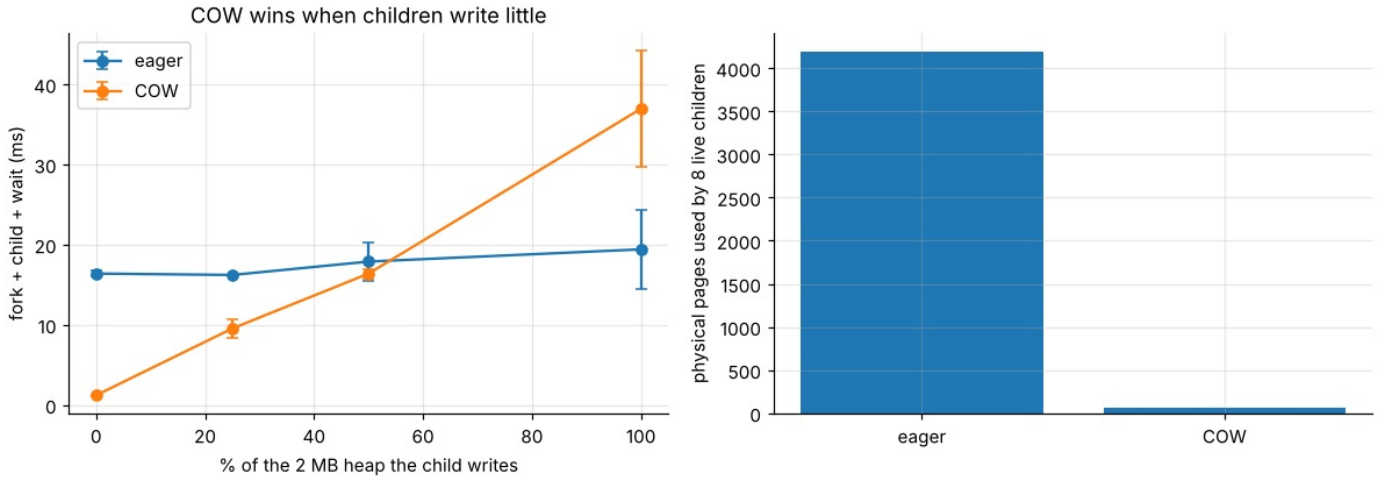
F_speedup



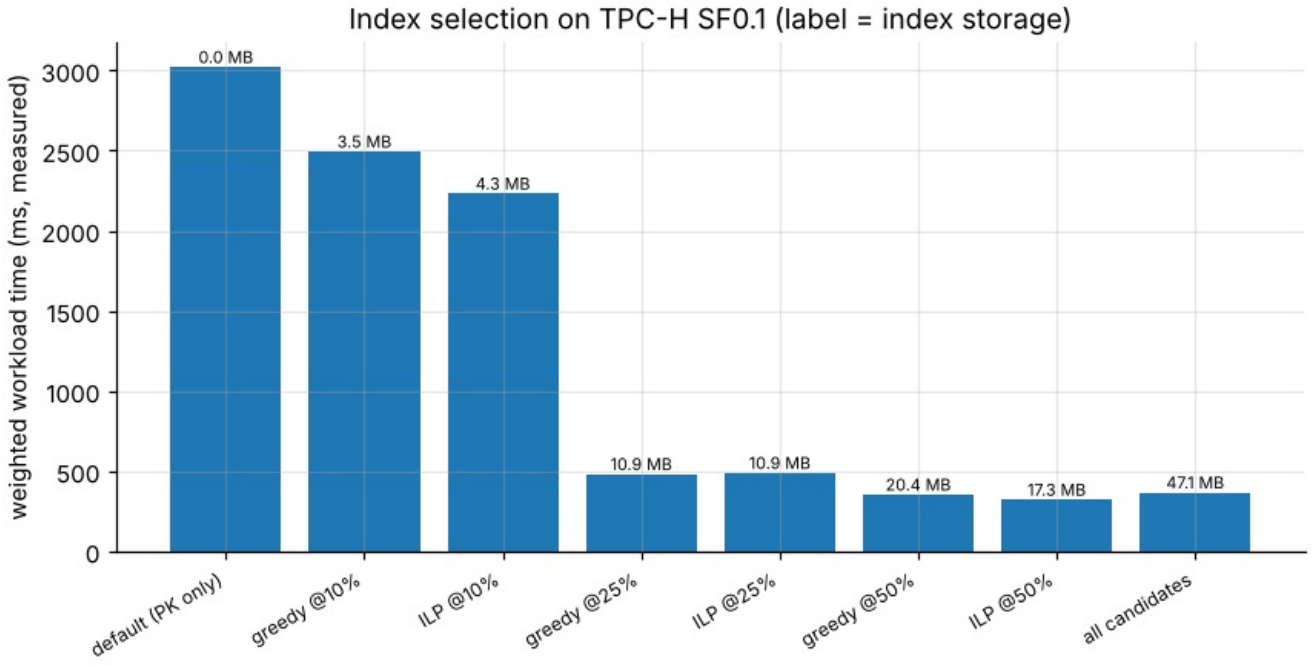
G_sched



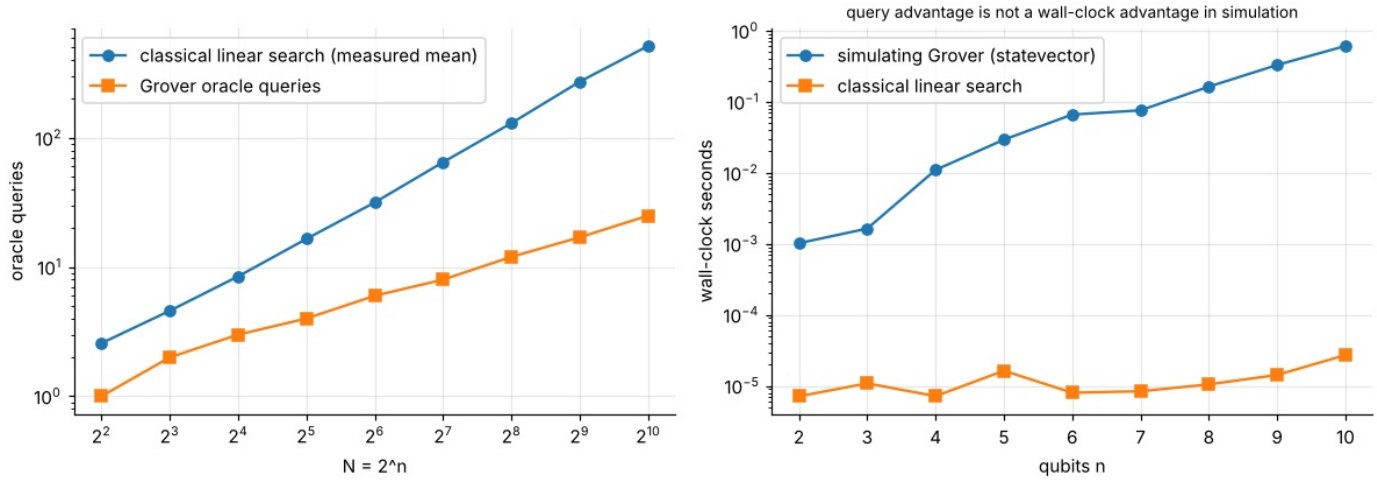
G_cow



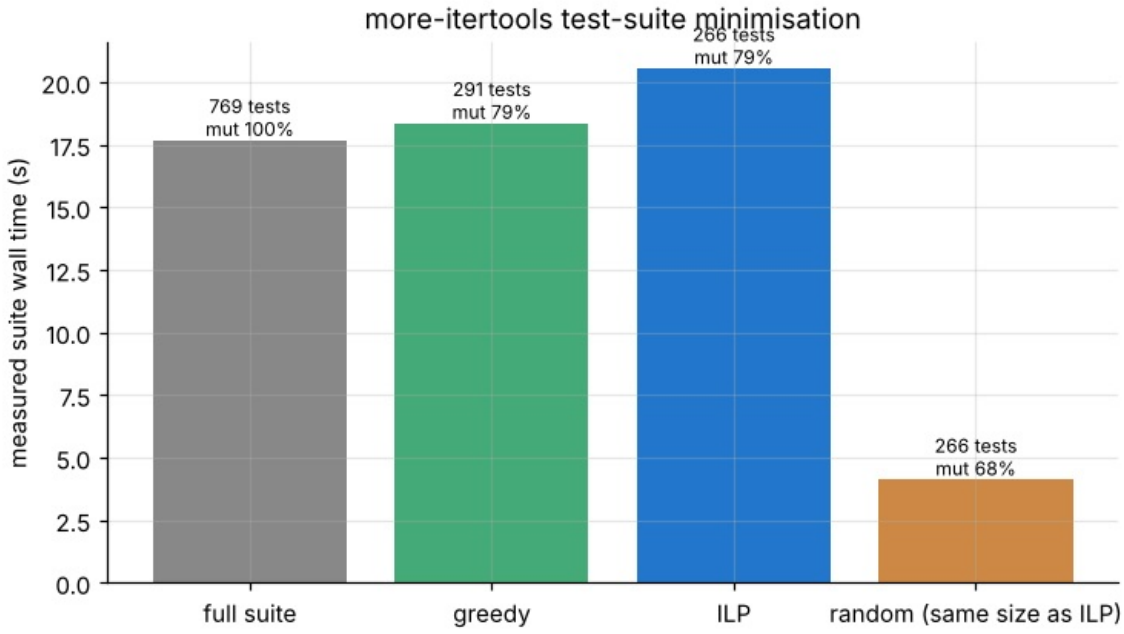
H_index_selection



I3_grover



J_minimization



Appendix B — Results summary

Track A — continuous optimisation

Iterations to f − f* < 1e-6 (tuned learning rate, same budget)

method	strongly convex quadratic (kappa=50)	convex quartic sum x^4	non-convex Rosenbrock
GD	214	546	–
Momentum	132	124	394
NAG	67	37	1426
AdaGrad	214	248	–
RMSProp	–	95	–
Adam	125	105	2208
Newton	1	9	5

GD with lr = 1/L obeys the (1−μ/L)^2k bound: **True**. Quartic (not strongly convex) GD log-log slope: **-1.95** (theory −2, sublinear).

Saddle point f = x² − y² + y⁴/4, start (0.5, 1e-4)

method	final x	converged to
GD	[0.0, 1.4142]	minimum
Momentum	[0.0, 1.4142]	minimum
Adam	[-0.0, 1.4142]	minimum
Newton	[0.0, 0.0]	saddle
Saddle-free Newton	H	

Fraction of 105 starts reaching the global minimum (tilted double well)

method	fraction global
GD	0.524
Momentum	0.619
NAG	0.571
Adam	0.524
RMSProp	0.524
AdaGrad	0.524

Noisy gradients (σ = 2, 10 seeds): final f

schedule	mean	std
SGD const lr=0.01	0.0239	0.0159
SGD const lr=0.001	0.0144	0.00597
SGD decay 1/(mu(k+k0))	0.002	0.00104
Adam lr=0.01	0.0106	0.00897
Momentum lr=0.002	0.0653	0.0564

Logistic regression (breast cancer), 30 epochs, 5 seeds

optimizer	batch	lr	f−f* mean	std	test acc
GD (full batch)	398	1.0	0.00243	0	0.959
SGD (batch=1)	1	0.05	0.00127	0.00231	0.956
Mini-batch SGD	32	0.3	0.000248	0.0003	0.957
Momentum	32	0.1	0.00032	0.00024	0.959
NAG	32	0.1	0.000255	0.000193	0.959
AdaGrad	32	0.3	0.00145	0.00268	0.957
RMSProp	32	0.01	0.000166	8.53e-05	0.958
Adam	32	0.01	0.000429	2.1e-05	0.959

Backprop gradient check (65-parameter MLP): relative error **4.5e-10**. Pearlmutter HVP error 6.4e-16; model Fisher vs Hessian 1.2e-17; empirical Fisher vs Hessian **0.74**; Hessian condition number 60.

method	iterations to 1e-8	final f−f*
GD (lr=1/L)	1638	9.99e-09
Diagonal-Hessian preconditioned GD	96	9.87e-09
Newton (exact H)	7	6.43e-13
Natural gradient (model Fisher)	7	6.43e-13
Empirical-Fisher preconditioning	–	0.00019
Hessian-free Newton-CG	7	6.43e-13
Low-rank (top-5) Newton	27	6.88e-09

Gauss-Newton converges in **7** iterations; Adam needs 273, GD 505 to match it. Best 4-of-12 features: exhaustive [0, 1, 3, 7] (SSE 28.28) vs Lasso relaxation [0, 1, 6, 7] (SSE 30.11).

Track B — discrete optimisation

0/1 knapsack (5 instances each; gap = % below optimum)

n items	greedy	DP	ILP	B&B	B&B nodes
20	1.054% / 0.04 ms	0.000% / 0.08 ms	0.000% / 10.26 ms	0.000% / 0.29 ms	92.8
50	0.244% / 0.06 ms	0.000% / 0.30 ms	0.000% / 24.66 ms	0.000% / 1.09 ms	255
100	0.098% / 0.20 ms	0.000% / 0.91 ms	0.000% / 92.61 ms	0.000% / 3.71 ms	893
200	0.031% / 0.25 ms	0.000% / 2.29 ms	0.000% / 73.47 ms	0.000% / 11.60 ms	2.2e+03

Makespan P||Cmax

case	LPT gap % mean	max	4/3 bound holds	ms LPT	ms ILP	ms B&B	B&B nodes
n=10,m=3	1.91	3.19	True	0.0282	44.9	0.293	65.6
n=14,m=4	4.13	7.5	True	0.0332	109	4.12	1.49e+03
n=20,m=5	2.46	3.88	True	0.0542	176	3.16e+03	9.69e+05

Max-Cut (n=8, 18 edges): QUBO/Ising/direct energy mismatch over all 2^n assignments = **0.0**; optimum 14.0; local search mean 13.15 (hits optimum 55%); SA 14.0; ILP 14.0.

penalty P	features chosen	feasible (=6)	= constrained optimum
0.0	3	False	False
0.02	4	False	False
0.05	5	False	False
0.1	5	False	False
0.2	5	False	False
0.5	5	False	False

0.5	5	False	False
1.0	6	True	True
2.0	6	True	True

Track C — quantum optimisation (simulator)

QAOA on the Track B Max-Cut (3 seeds)

depth	optimizer	expected-cut ratio	P(optimal cut)	circuit evaluations
p=1	COBYLA	0.758	0.0583	49.7
p=1	param-shift GD	0.717	0.0372	2.08e+03
p=1	SPSA	0.771	0.0652	450
p=2	COBYLA	0.785	0.111	199
p=2	param-shift GD	0.766	0.151	4.16e+03
p=2	SPSA	0.759	0.0875	450
p=3	COBYLA	0.723	0.0626	200
p=3	param-shift GD	0.766	0.12	6.24e+03
p=3	SPSA	0.748	0.0468	450

Classical baselines: random 0.643, local search 0.939, SA 1.0. Parameter-shift vs finite difference error 1.1e-09.

noise level	ratio	P(optimal)
noiseless	0.838	0.22
low (1e-4,1e-3,1%)	0.827	0.203
medium (1e-3,1e-2,2%)	0.801	0.153
high (5e-3,5e-2,5%)	0.721	0.052

shots	std of energy	std·√shots
16	0.49	1.96
64	0.278	2.23
256	0.129	2.07
1024	0.0485	1.55
4096	0.0251	1.6

VQE for H2 (5 seeds)

optimizer	final error (Ha)	iters to chem. acc.	runs reaching
param-shift GD	8e-05	17.8	5/5
SPSA	0.0026	67.5	2/5
QNG	-1.02e-15	4	5/5
COBYLA	2.94e-07	44.8	5/5

Barren plateau: Var[$\partial E/\partial \theta_1$]

qubits	global cost	local cost
2	0.182	0.389
4	0.0666	0.248
6	0.0199	0.17
8	0.00546	0.179
10	0.00138	0.0883

Track D — submodular, parallel, distributed

Coverage property test: 0 violations / 3000; complementary objective: 209 violations (guarantee not claimed).

k	greedy/OPT mean	min	≥ 1−1/e	calls greedy	calls lazy
3	0.995	0.978	True	63	28.4
4	0.97	0.939	True	81	36.2
5	0.971	0.948	True	98	40.8

Large instance (300 sites, k=25): greedy 7225 oracle calls vs lazy 494, same value: True.

Scheduling heavy-tailed tasks, measured (pthreads, 2 CPUs)

strategy	1 thread	2 threads	4 threads (oversubscribed)
block	1.00× (imb 1.00)	1.15× (imb 1.76)	1.30× (imb 2.55)
cyclic	0.99× (imb 1.00)	1.96× (imb 1.01)	1.96× (imb 1.05)
central	0.99× (imb 1.00)	2.02× (imb 1.00)	1.99× (imb 1.04)
steal	1.00× (imb 1.00)	1.97× (imb 1.00)	1.99× (imb 1.04)

Karp-Flatt serial fraction from the 2-core measurement: raw 0.0163 → within timing noise of 0 on 2 cores (repeat runs gave -0.016 and +0.016). Model predictions (not measured) for assumed serial fractions:

serial fraction	Amdahl p=16	Amdahl p=64	Gustafson p=16	Gustafson p=64
0.01	13.9×	39.3×	15.8×	63.4×
0.05	9.1×	15.4×	15.2×	60.9×
0.1	6.4×	8.8×	14.5×	57.7×

strategy (simulated)	1	2	4	8	16
block	1.00	1.13	1.44	1.93	2.75
cyclic	1.00	1.99	3.92	7.61	14.30
central	1.00	1.98	3.91	7.58	14.24
steal	1.00	1.99	3.94	7.67	14.68

n	ijk s	ikj s	blocked s	agree
256	0.0195	0.0112	0.0149	True
512	0.422	0.114	0.159	True
800	0.76	0.333	0.405	True

sync	1 thread s	2 threads s
mutex	0.0812	0.215
atomic	0.0262	0.0582
local	0.00178	0.0011

Data parallel (2 procs): speedup 1.54×, gradient error 6.8e-13. Tensor parallel: 1.25×, error 0.0. Measured 2-stage pipeline: M=1 0.25×, M=4 0.53×, M=16 0.82×.

FedAvg (10 clients, 5 seeds; target f − f* ≤ 2e-3)

setting	final f−f*	rounds to target	runs
centralised_minibatch_SGD(same total gradient steps as FedAvg E=5)	0.00157	–	–
IID E=1	0.0775	–	0/5
IID E=5	0.0145	–	0/5
IID E=20	0.000398	37.2	5/5
IID E=50	0.000163	15	5/5
non-IID E=1	0.0776	–	0/5
non-IID E=5	0.0173	–	0/5
non-IID E=20	0.00271	50.5	2/5
non-IID E=50	0.00241	25.5	2/5

Two-task model (validation MSE / target variance)

weighting	task 1	task 2	mean
equal	0.0228 ± 0.01	9.79e-05 ± 2.4e-05	0.0114
MGDA	0.00702 ± 0.0053	0.207 ± 0.015	0.107
GradNorm	0.00663 ± 0.0031	0.00764 ± 0.0014	0.00714

Track E — nurse rostering (mean of 5 instances per ward size)

nurses	method	penalty	understaffed	no senior	requests worked	night spread	hard viol.	secs
10	Manual baseline	1996	8.4	15.8	16.6	8.4	0	0.00074
10	Greedy	714	3.2	4.6	11	1.6	0	0.0043
10	Simulated annealing	291.2	0.4	1.6	13.6	3.6	0	1.49

10	Simulated annealing	291.2	0.4	1.0	15.0	3.0	0	1.49
10	ILP (HiGHS)	146.8	0.4	1.2	3.4	0	0	0.882
20	Manual baseline	2458	13	12.8	35.8	6.4	0	0.00172
20	Greedy	448	2.4	0.4	17.8	1.2	0	0.0136
20	Simulated annealing	185.8	0.2	0	14.8	3.4	0	1.91
20	ILP (HiGHS)	37	0	0	3.6	0.2	0	1.7
40	Manual baseline	4179	31.2	6.4	64.6	5.8	0	0.00282
40	Greedy	911	6	0.2	29.4	1	0	0.0577
40	Simulated annealing	413	0.2	0	36.4	5.8	0	2.14
40	ILP (HiGHS)	8	0	0	0.8	0	0	3.59
80	Manual baseline	7659	59.8	3.4	144	7	0	0.00477
80	Greedy	1944	12	0	73.8	1.2	0	0.214
80	Simulated annealing	1205	0.8	0	110	5.4	0	2.52
80	ILP (HiGHS)	52	0	0	5.2	0	0	10.1

+25% demand: x1.0: Manual cost 2335 understaffed 12, ILP cost 0 understaffed 0; x1.25: Manual cost 5665 understaffed 47, ILP cost 4670 understaffed 46

Track F — compiler

program	output same	dyn. instr	cycles	static size	native (gcc -O0)	pass ms
sum_invariant	True	30013 → 18005	38013 → 18005	26 → 12	0.90×	1.0
cse_poly	True	51009 → 21005	66009 → 24005	41 → 17	1.95×	2.0
primes	True	60931 → 49034	161078 → 149181	41 → 23	1.19×	2.0
matmul	True	189869 → 124645	541869 → 460685	53 → 37	1.55×	4.3
fib_const	True	39027 → 27008	102054 → 84027	38 → 15	0.94×	1.5

configuration	total cycles (5 programs)
none	909023
only constprop	899017
only copyprop	909023
only cse	909023
only dce	897023
only licm	788484
only sr	909023
default pipeline (fixpoint)	735903
default minus constprop	753886
default minus copyprop	768403
default minus cse	746403
default minus dce	778377
default minus licm	759843
default minus sr	742905

Pass ordering (L=4, 1296 sequences): exhaustive best ['licm', 'copyprop', 'dce', 'licm'] = 753981 cycles (1296 evaluations); greedy ['licm', 'dce', 'licm', 'licm'] = 777888 (24 evaluations); median sequence 792772; no optimisation 909023.

random interference graphs (10 each)	Chaitin excess spill cost	suboptimal instances	Chaitin ms	ILP ms	ILP optimal
n=30,p=0.2,K=3	184%	10	0.187	594	10
n=30,p=0.3,K=4	375%	10	0.22	1.88e+03	10
n=30,p=0.4,K=5	496%	10	0.23	4.11e+03	10

Differential fuzzing: 1000 random programs × 3 pipelines (+ register allocation): **0 mismatches**.

Track G — xv6 (QEMU, 1 CPU for benchmarks; ticks ≈ 0.1 s)

workload	policy	avg turnaround	avg response	makespan	per kind (turnaround/response)
W1 mixed batch (6 CPU jobs arrive together)	RR (original)	11.3 ± 0.67	2.17 ± 0	19.7 ± 1.2	long_turnaround=16.6, long_response=1.7, short_turnaround=6.1, short_response=2.7
W1 mixed batch (6 CPU jobs	MLFQ	12.6 ± 0.63	2.17 ± 0	20 ± 1.7	long_turnaround=18.0, long_response=1.7,

arrive together)					short_turnaround=7.1, short_response=2.7
W2 2 CPU hogs + 3 interactive	RR (original)	27.7 ± 0.5	1.4 ± 2.7e-16	36.3 ± 1.5	hog_turnaround=36.2, hog_response=0.5, interactive_turnaround=22.0, interactive_response=2.0
W2 2 CPU hogs + 3 interactive	MLFQ	22.5 ± 1.7	1.4 ± 2.7e-16	37.7 ± 3.8	hog_turnaround=36.7, hog_response=0.5, interactive_turnaround=13.0, interactive_response=2.0
W3 short jobs arriving behind 2 long jobs	RR (original)	8.42 ± 0.51	1.58 ± 0.072	30 ± 2	long_turnaround=27.5, long_response=0.5, short_turnaround=2.1, short_response=1.9
W3 short jobs arriving behind 2 long jobs	MLFQ	6.83 ± 0.26	0.125 ± 0	28.3 ± 0.58	long_turnaround=27.2, long_response=0.5, short_turnaround=0.1, short_response=0.0

COW isolation test: {'0': ['PASS', 'PASS', 'PASS'], '1': ['PASS', 'PASS', 'PASS']}.

fork+exit, 2 MB heap	µs per fork	pages copied / 20 forks
eager copy (original)	1.79e+04 ± 1.4e+03	1.03e+04 ± 0
copy-on-write	1.12e+03 ± 18	0 ± 0

child writes	eager µs	COW µs
0%	1.64e+04 ± 3.3e+02	1.21e+03 ± 83
25%	1.62e+04 ± 21	9.54e+03 ± 1.2e+03
50%	1.79e+04 ± 2.4e+03	1.64e+04 ± 5.7e+02
100%	1.94e+04 ± 5e+03	3.69e+04 ± 7.2e+03

8 live children	pages used
eager	4.19e+03 ± 0
COW	57 ± 0

Track H — PostgreSQL 16, TPC-H SF 0.1 (official dbgen)

configuration	weighted workload ms (measured)	index MB	build s	results identical	# indexes
default (PK only)	3021.5	0.0	0.00	True	0
greedy @10%	2494.2	3.5	0.31	True	6
ILP @10%	2232.1	4.3	0.22	True	1
greedy @25%	481.7	10.9	0.62	True	7
ILP @25%	489.3	10.9	0.64	True	7
greedy @50%	353.9	20.4	1.12	True	11
ILP @50%	322.0	17.3	1.59	True	9
all candidates	367.0	47.1	2.94	True	16

budget (of all candidates)	greedy what-if saving	greedy what-if calls	ILP what-if saving	ILP solve
10%	23.5%	50	30.5%	8 ms
25%	78.8%	62	78.8%	7 ms
50%	81.8%	114	82.0%	9 ms

Track I — quantum algorithms (simulated; FakeManilaV2 noise model for the device run)

Deutsch–Jozsa n=5: all correct = True with 1 query (classical worst case 17). Bernstein–Vazirani: True with 1 query vs 6. QFT vs DFT matrix error 3.8e-15. Shor N=15: factored for a = 7: True, 2: True, 4: True, 11: True, 13: True.

n	N	Grover queries	classical queries	P(success)	CX gates	sim s	classical s
2	4	1	2.58	1	2	0.00102	7.15e-06
3	8	2	4.59	0.945	24	0.00164	1.09e-05
4	16	3	8.47	0.961	84	0.011	7.18e-06
5	32	4	16.5	0.999	288	0.0293	1.64e-05
6	64	6	31.6	0.997	984	0.0661	8.04e-06
7	128	8	64.8	0.996	1952	0.0758	8.41e-06
8	256	12	130	1	4080	0.163	1.05e-05
9	512	17	272	0.999	7684	0.33	1.43e-05
10	1024	25	514	0.999	14500	0.611	2.72e-05

Grover n=3 on FakeManilaV2 noise model: P(marked) 0.562 vs ideal 0.937 (random 0.125), 37 two-qubit gates after routing.

counting qubits	phi=0.312500	phi=0.333333
3	0.0625	0.0417
4	0	0.0208
5	0	0.0104
6	0	0.00521
7	0	0.0026
8	0	0.0013

Adiabatic annealing (8 qubits, min gap 0.581): T=0.5: P(opt)=0.011, T=2: P(opt)=0.085, T=5: P(opt)=0.295, T=10: P(opt)=0.574, T=20: P(opt)=0.928, T=40: P(opt)=0.997.

Embedding on Chimera C(4) 128 qubits: Track B Max-Cut (8 nodes): 16 qubits, max chain 3; K6: 14 qubits, max chain 3; K10: 34 qubits, max chain 4; K14: 66 qubits, max chain 6; K18: FAILED; K24: FAILED; K32: FAILED.

Embedding on Pegasus P(6) 680 qubits: Track B Max-Cut (8 nodes): 9 qubits, max chain 2; K6: 8 qubits, max chain 2; K10: 18 qubits, max chain 2; K14: 34 qubits, max chain 3; K18: 48 qubits, max chain 3; K24: 83 qubits, max chain 5; K32: 131 qubits, max chain 5.

model (moons, 5 splits)	test accuracy	train s	params	circuit evaluations
VQC (data re-uploading, 12 params)	0.933 ± 0.070	21.7	12	1.26e+05
Quantum kernel SVM (ZZ map)	0.778 ± 0.073	0.00822	n_support (dual)	6.51e+03
Logistic regression	0.883 ± 0.030	0.00242	3	0
RBF-SVM	0.972 ± 0.034	0.00121	n_support (dual)	0
MLP (16 hidden)	0.867 ± 0.030	0.171	65	0

Track J — test-suite minimisation, more-itertools @ 08dca284 (769 tests, 1970 covered lines, 100.0% coverage)

suite	tests	Σ test time s	measured wall s	line coverage kept	mutation score
full suite	769	104	17.7	100.0%	100.0%
greedy	291	94.1	18.4	100.0%	78.6%
ILP	266	92.3	20.5	100.0%	78.6%
random (same size as ILP)	266	16.6	4.11	69.5%	67.9%

Mutants: 40 generated, 28 killed by the full suite. APFD: original order 0.506, random order 0.709, greedy additional coverage/time 0.781. Bottleneck (top 3 tests by time): test_primes 65.0%; test_concurrent_consumers 13.4%; test_nth_prime_approximate 6.9%.